

JAVA

## 1. Explain Inheritance.

**A:** Inheritance in Java is a feature where one class acquires the properties and methods of another class.

### It helps in:

- Code reusability
- Reducing duplicate code
- Creating a relationship between classes

### Syntax:

```
class Parent {  
    // properties and methods  
}  
class Child extends Parent {  
    // additional properties and methods  
}
```

### Types of Inheritance in Java:

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance

### Advantages of Inheritance:

- Reuse existing code
  - Easier maintenance
  - Better organization
  - Supports polymorphism
- 

## 2. Extends:

**A:** `extends` is used to create inheritance between classes. It allows a child class to inherit fields and methods from a parent class.

**Example:**

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound(); // inherited method  
        d.bark();  
    }  
}
```

---

**3. Super():**

**A:** `super()` is used to call the parent class constructor. It is usually written inside the child class constructor.

**Example:**

```
class Animal {  
    Animal() {
```

```
        System.out.println("Animal constructor");
    }
}
class Dog extends Animal {
    Dog() {
        super(); // calls parent constructor
        System.out.println("Dog constructor");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```

---

#### 4. Abstract Class in Java:

**A:** An abstract class is a class that cannot be instantiated directly. It is used as a blueprint for other classes.

- Declared using abstract keyword
- Can contain:
  - abstract methods
  - normal methods

#### **Abstract Method:**

A method without a body.

**Ex:** abstract void sound();

**Example:**

```
abstract class Animal {  
    // abstract method  
    abstract void sound();  
    // normal method  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    // implementing abstract method  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
        d.eat();  
    }  
}
```

---

**5. Final in Java:**

**A:** The final keyword is used to restrict modification.

**It can be used with:**

- Variables
- Methods
- Classes

**1. Final Variable:**

A final variable becomes a constant. Its value cannot be changed after assignment.

**Example:**

```
public class Main {  
    public static void main(String[] args) {  
        final int AGE = 18;  
        System.out.println(AGE);  
        // AGE = 20; // Error  
    }  
}
```

**2. Final Method:**

A final method cannot be overridden by child classes.

**Example:**

```
class Animal {  
    final void show() {  
        System.out.println("Final method");  
    }  
}  
  
class Dog extends Animal {  
    // Error if overridden
```

```

    /*
    void show() {
        System.out.println("Dog method");
    }
    */
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.show();
    }
}

```

### **3. Final Class:**

A final class cannot be inherited.

#### **Example:**

```

final class Animal {
    void show() {
        System.out.println("Final class");
    }
}

// Error

/*

class Dog extends Animal {
}

*/

```

```
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Animal();  
        a.show();  
    }  
}
```

---

## 6. Packages in Java:

**A:** A **package** is a collection of related classes and interfaces.

### It is used to:

- Organize code
- Avoid name conflicts
- Provide access protection

### Types of Packages:

**1. Built-in Packages:** Provided by Java.

#### Examples:

- java.util
- java.io
- java.lang

**2. User-defined Packages:** Created by programmer.

---

## 7. Threads in Java:

**A:** A thread is a lightweight process. It allows Java programs to perform multiple tasks simultaneously.

### Creating Threads:

### **1. Extending Thread class:**

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

### **2. Implementing Runnable:**

```
class MyThread implements Runnable {  
    public void run() {  
        System.out.println("Runnable thread");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Thread t = new Thread(new MyThread());  
        t.start();  
    }  
}
```

## 8. States of Thread in Java:

**A:** A thread passes through different states during execution.

### Thread Life Cycle:

NEW → RUNNABLE → RUNNING → WAITING/BLOCKED → TERMINATED

**1. NEW:** Thread object created.

Thread t = new Thread();

**2. RUNNABLE:** After calling start().

t.start();

**3. RUNNING:** Thread is executing run() method.

### **4. WAITING / BLOCKED**

Thread waits for:

- another thread
- resource
- sleep time

Thread.sleep(1000);

**5. TERMINATED:** Execution completed.

---

## 9. Applets in Java:

**A:** An Applet is a small Java program that runs inside a web browser or applet viewer. Applet classes are available in: java.applet

### Features of Applets:

- Runs inside browser/applet viewer
- No main() method
- Uses AWT for GUI
- Event-driven

## 10. IOException in Java:

**A:** IOException occurs when input/output operations fail.

### Example:

```
import java.io.*;

public class Main {

    public static void main(String[] args) {

        try {

            FileReader fr = new FileReader("test.txt");

            int ch;

            while((ch = fr.read()) != -1) {

                System.out.print((char) ch);

            }

            fr.close();

        }

        catch(IOException e) {

            System.out.println("IO Exception occurred");

        }

    }

}
```

### Exceptions in Java:

An **exception** is an unwanted event that interrupts program execution.

### Types of Exceptions:

**1. Checked Exceptions:** Checked at compile time.

### Examples:

- IOException

- SQLException

**Must be handled using:**

- try-catch
- throws

**2. Unchecked Exceptions:** Occur at runtime.

**Examples:**

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException

**Example:** `int a = 10 / 0;`

**3. Errors:** Serious problems not usually handled by program.

**Examples:**

- OutOfMemoryError
  - StackOverflowError
- 

## **11. Delegation Event Model in Java:**

**A:** The Delegation Event Model is used in Java GUI programming to handle events. Introduced in AWT and Swing.

**What is an Event?**

An event is an action like:

- Mouse click
- Key press
- Button click

## **Main Components of Delegation Event Model:**

**1. Event Source:** The object that generates the event.

### **Examples:**

- Button
- TextField
- Checkbox

Example:

```
Button b = new Button("Click");
```

**2. Event Object:** Contains information about the event.

### **Examples:**

- ActionEvent
- MouseEvent
- KeyEvent

**3. Event Listener:** Receives and handles the event.

### **Listener interfaces:**

- ActionListener
- MouseListener
- KeyListener